

Project Title:

Real-Time Object Detection Web Application Using YOLOv5 and Streamlit

Abstract :

This project presents the development of a user-friendly web application for real-time object detection using the YOLOv5 deep learning model. The app is built using Streamlit, a lightweight Python framework that simplifies the creation of interactive web interfaces for machine learning projects. Users can upload images through the interface, upon which the application performs object detection using a pretrained YOLOv5s model.

The detection process involves converting the uploaded image into a suitable format, feeding it to the YOLOv5 model, and visualizing the results with bounding boxes and confidence scores. The integration of OpenCV and PIL enables precise image handling and annotation.

This project demonstrates the power of combining cutting-edge deep learning techniques with rapid web deployment tools to create practical and accessible computer vision applications. It serves as a foundational model that can be extended to real-time video detection, edge-device deployment, or customized object classes for domain-specific use cases.

Tech Stack Used

- **Programming Language:**
 - Python 3.10+
- **Deep Learning Framework:**
 - PyTorch
- **Object Detection Model:**
 - YOLOv5s (pre-trained on the COCO dataset)
- **Model Source:**

- Ultralytics YOLOv5 (cloned from GitHub)
- **Computer Vision Libraries:**
 - OpenCV (for image processing and drawing bounding boxes)
 - Pillow (PIL) for image loading and format conversion
- **Numerical Computation:**
 - NumPy
- **Visualization Libraries (used internally by YOLOv5):**
 - Matplotlib
 - Seaborn
- **Web Application Framework:**
 - Streamlit (for building the interactive web interface)

Introduction

Object detection is a crucial aspect of computer vision, enabling machines to identify and locate objects within an image or video. With the advancement of deep learning, real-time object detection has become highly accurate and practical for deployment in various domains such as surveillance, autonomous vehicles, healthcare, retail analytics, and more.

This project leverages YOLOv5 (You Only Look Once) — a state-of-the-art, real-time object detection algorithm — and integrates it with a Streamlit-based web interface. The application allows users to upload an image, and it automatically detects and labels all identifiable objects using a pretrained YOLOv5 model.

Code:

```
# YOLO_DETECTION.py  
  
# !pip install streamlit opencv-python pillow torch torchvision torchaudio  
  
# Clone YOLOv5 once, not every time
```

```
# !git clone https://github.com/ultralytics/yolov5.git

# Import libraries

import streamlit as st

import torch

import numpy as np

import cv2

from PIL import Image

import os

import sys

# Append YOLOv5 repo to path if running from cloned repo

YOLO_DIR = 'yolov5'

if YOLO_DIR not in sys.path:

    sys.path.append(YOLO_DIR)


# Load model (make sure yolov5 directory exists)

model = torch.hub.load('D:/INTERNSHIP PROJECT1/yolov5', 'yolov5s',
source='local') # uses yolov5s model

model.eval()


# Streamlit UI

st.title("🚀 YOLOv5 Object Detection Web App")

st.write("Upload an image and see YOLOv5 detect objects in it.")


# File uploader

uploaded_file = st.file_uploader("Upload Image", type=['jpg', 'jpeg', 'png'])
```

```

# When file is uploaded
if uploaded_file is not None:
    image = Image.open(uploaded_file).convert("RGB")
    st.image(image, caption='Uploaded Image', use_column_width=True)

if st.button("Detect Objects"):
    # Convert PIL image to NumPy
    img_array = np.array(image)

    # Convert RGB to BGR for OpenCV
    img_bgr = cv2.cvtColor(img_array, cv2.COLOR_RGB2BGR)

    # Perform detection
    results = model(img_bgr)
    detections = results.pandas().xyxy[0]

    # Draw bounding boxes and labels
    for _, row in detections.iterrows():
        x1, y1, x2, y2 = map(int, [row['xmin'], row['ymin'], row['xmax'],
row['ymax']])

        label = f'{row['name']} {row['confidence']:.2f}'
        cv2.rectangle(img_array, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.putText(img_array, label, (x1, y1 - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 0, 0), 2)

    # Show detected image

```

```
st.image(img_array, caption='Detected Image', channels="RGB",  
use_column_width=True)
```

Output:

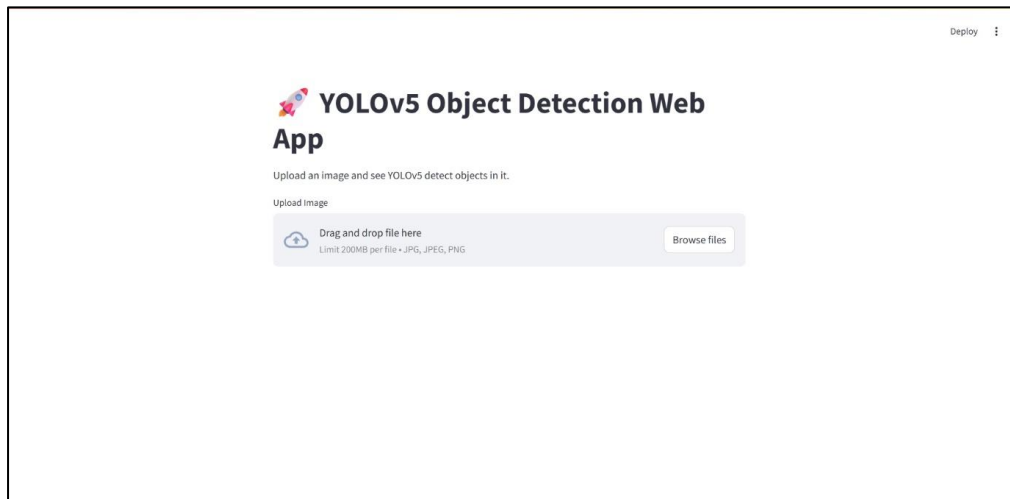


FIG: WEB APP INTERFACE

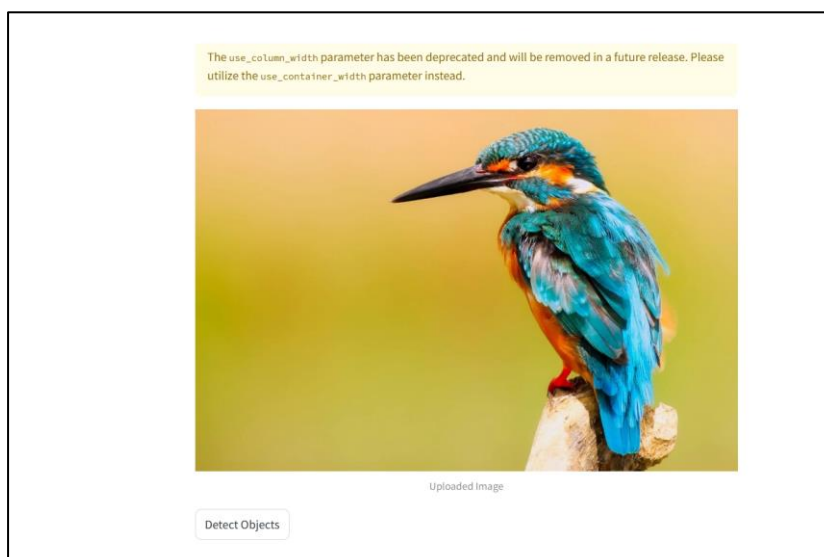


FIG: UPLOADED IMAGE

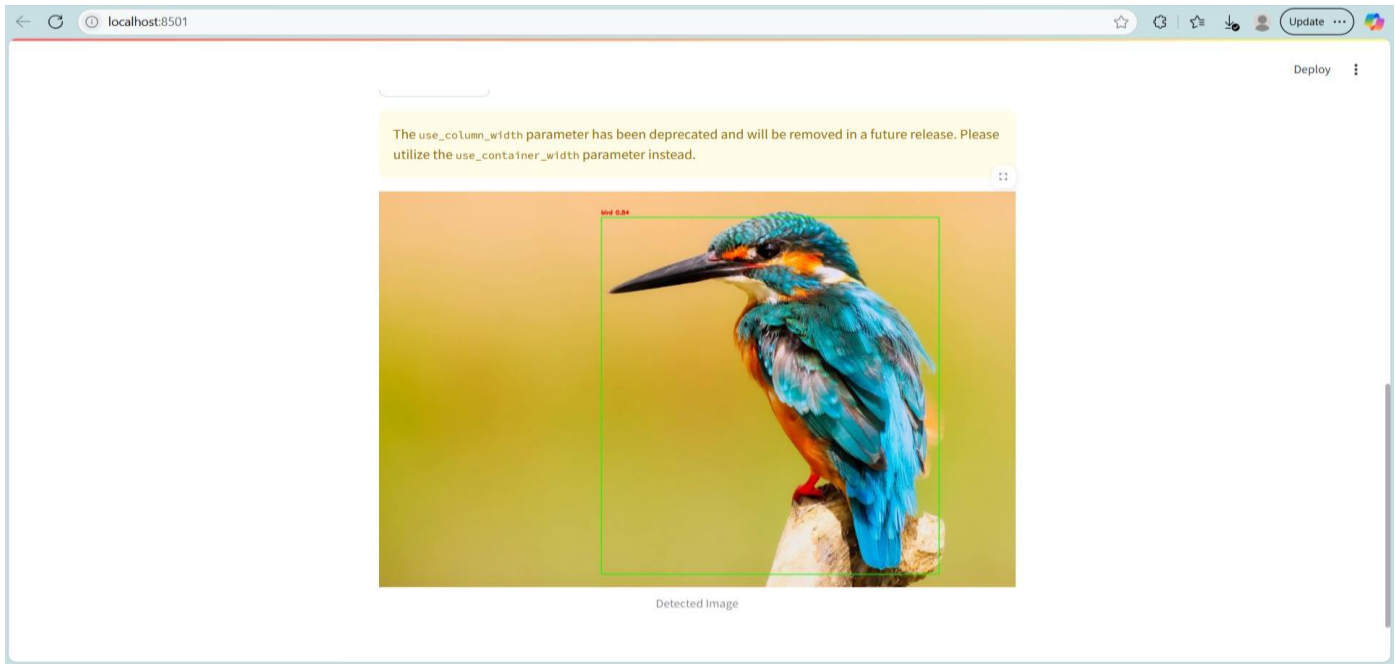


FIG: DETECTED IMAGE

Conclusion

This project successfully demonstrates the development of an interactive, real-time object detection web application using YOLOv5 and Streamlit. By integrating deep learning with a user-friendly UI, the system enables users to upload any image and automatically detect multiple object classes with high accuracy and speed.

The project shows how complex AI models like YOLO can be deployed and made accessible through lightweight web technologies. It highlights the practical utility of combining deep learning with modern Python-based tools to build intelligent and scalable computer vision solutions.

This system can be extended further for:

- Real-time video feed detection
- Custom object training
- Edge deployment (Jetson Nano, Raspberry Pi)
- Integration into surveillance systems or mobile apps